

# PROJECT-1

## AI BASED FAULT DETECTION IN DIGITAL CIRCUIT

**The core idea** — digital circuits can have manufacturing defects or runtime failures. The most common model is "stuck-at faults" (a signal stuck at 0 or 1 regardless of input). Your AI model learns to detect these by analyzing the difference between expected and actual circuit outputs.

Basic code of full adder in python

```
import pandas as pd

data = []

truth_table = [
    [0,0,0,0,0],
    [0,0,1,1,0],
    [0,1,0,1,0],
    [0,1,1,0,1],
    [1,0,0,1,0],
    [1,0,1,0,1],
    [1,1,0,0,1],
    [1,1,1,1,1]
]

for row in truth_table:
    A,B,Cin,Sum,Cout = row

    # No fault
    data.append([A,B,Cin,Sum,Cout,"No Fault"])

    # Sum stuck at 0
    data.append([A,B,Cin,0,Cout,"Sum Stuck-at-0"])

    # Sum stuck at 1
    data.append([A,B,Cin,1,Cout,"Sum Stuck-at-1"])

    # Cout stuck at 0
    data.append([A,B,Cin,Sum,0,"Cout Stuck-at-0"])
```

```

# Cout stuck at 1
data.append([A,B,Cin,Sum,1,"Cout Stuck-at-1"])

df = pd.DataFrame(data, columns=[
    "A","B","Cin","Sum","Cout","Fault"
])

df.to_csv("dataset.csv", index=False)

print(df.head())
print("Dataset created successfully!")

```

## CHECKING ACCURACY (1.0)

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import joblib

# Load dataset
df = pd.read_csv("dataset.csv")

# Features and labels
X = df[["A","B","Cin","Sum","Cout"]]
y = df["Fault"]

# Train model on full dataset
model = DecisionTreeClassifier(random_state=42)

model.fit(X, y)

# Predict on same data
y_pred = model.predict(X)

# Accuracy
accuracy = accuracy_score(y, y_pred)

print("Accuracy:", accuracy)

```

## USER INPUT AND THEN COMPUTER TELLING FAULT

```

import joblib
import pandas as pd

# Load trained model
model = joblib.load("fault_model.pkl")

# User input

```

```
A = int(input("Enter A: "))
B = int(input("Enter B: "))
Cin = int(input("Enter Cin: "))
Sum = int(input("Enter observed Sum: "))
Cout = int(input("Enter observed Cout: "))

# Create dataframe with feature names
input_data = pd.DataFrame(
    [[A,B,Cin,Sum,Cout]],
    columns=["A","B","Cin","Sum","Cout"]
)

# Prediction
prediction = model.predict(input_data)

print("Detected Fault:", prediction[0])
```

We are building a system that:

1. Takes circuit inputs
2. Observes outputs
3. Checks whether outputs are correct
4. Detects faults automatically using ML

It is like an intelligent testing system for digital circuits.

---

## STEP 1 — Understanding Full Adder

A full adder adds 3 binary bits:

- A
- B
- Cin (Carry input)

Outputs:

- Sum
  - Cout (Carry output)
- 

## Example

Suppose:

$$A = 1$$

$$B = 0$$

$$C_{in} = 1$$

Binary addition:

$$1 + 0 + 1 = 10$$

In binary:

- right digit  $\rightarrow$  Sum
- left digit  $\rightarrow$  Carry

So:

$$\text{Sum} = 0$$

$$\text{Cout} = 1$$

---

## FULL ADDER EQUATIONS

**Sum Equation**

$$\text{Sum} = A \oplus B \oplus C_{in} \quad \text{Sum} = A \oplus B \oplus C_{in} \quad \text{Sum} = A \oplus B \oplus C_{in}$$

XOR means:

- same bits  $\rightarrow 0$
  - different bits  $\rightarrow 1$
- 

## Carry Equation

$$C_{out} = AB + BC_{in} + AC_{in}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

Meaning:

Carry becomes 1 if at least two inputs are 1.

---

## STEP 2 — Truth Table

This is the correct behavior of a full adder.

| A | B | C <sub>i</sub> | S <sub>u</sub> | C <sub>o</sub> |
|---|---|----------------|----------------|----------------|
|   |   | n              | m              | ut             |
| 0 | 0 | 0              | 0              | 0              |
| 0 | 0 | 1              | 1              | 0              |
| 0 | 1 | 0              | 1              | 0              |
| 0 | 1 | 1              | 0              | 1              |

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

This table is the “ideal circuit.”

---

## STEP 3 — Understanding Faults

Real circuits can fail.

Example:

- wire broken
- transistor damaged
- short circuit
- output permanently fixed

One common fault:

### Stuck-at Fault

Meaning:

Output gets stuck permanently at:

- 0
- OR

- 1
- 

## Example of Fault

Correct output:

Sum = 0

But due to fault:

Sum = 1

This is:

Sum Stuck-at-1

---

## STEP 4 — Dataset Creation

This part:

```
data.append([A,B,Cin,Sum,Cout,"No Fault"])
```

means:

Store:

- inputs
- outputs
- label

Example row:

| <b>A</b> | <b>B</b> | <b>Ci</b> | <b>Su</b> | <b>Co</b> | <b>Fault</b> |
|----------|----------|-----------|-----------|-----------|--------------|
|          |          | <b>n</b>  | <b>m</b>  | <b>ut</b> |              |
| 1        | 0        | 1         | 0         | 1         | No<br>Fault  |

---

## Faulty Rows

Suppose the correct Sum is 0.

This line:

```
data.append([A,B,Cin,1,Cout,"Sum Stuck-at-1"])
```

creates artificial faulty data.

Example:

| <b>A</b> | <b>B</b> | <b>Ci</b> | <b>Su</b> | <b>Co</b> | <b>Fault</b>          |
|----------|----------|-----------|-----------|-----------|-----------------------|
|          |          | <b>n</b>  | <b>m</b>  | <b>ut</b> |                       |
| 1        | 0        | 1         | 1         | 1         | Sum<br>Stuck-at-<br>1 |

Meaning:

Circuit incorrectly produced Sum=1.

---

## WHY WE CREATE MANY ROWS

Machine learning learns patterns.

We show examples like:

- “When this output changes this way → this fault occurs.”

This is training.

---

## **STEP 5 — Understanding Features and Labels**

This code:

```
X = df[["A","B","Cin","Sum","Cout"]]
```

means:

These are INPUT FEATURES for AI.

AI observes:

- A
  - B
  - Cin
  - observed Sum
  - observed Cout
- 

This line:

```
y = df["Fault"]
```

means:

This is the ANSWER AI should learn.

Example:

Inputs → Fault Type

---

## **STEP 6 — Training the Model**

`model.fit(X, y)`

This is where learning happens.

The Decision Tree studies patterns like:

IF:

Expected Sum should be 0  
but observed Sum is 1

THEN:

Likely Sum Stuck-at-1

---

## **STEP 7 — Prediction Phase**

User enters:

A = 1

B = 0

Cin = 1

Observed Sum = 1  
Observed Cout = 1

Program asks:

**“Is this output correct?”**

Correct output should be:

Sum = 0  
Cout = 1

But observed Sum became 1.

So model predicts:

Sum Stuck-at-1

---

## **WHAT ARE “OBSERVED OUTPUTS”?**

VERY IMPORTANT.

These values:

Observed Sum  
Observed Cout

represent:

**actual outputs coming from the real circuit.**

Maybe:

- from hardware
- from simulation
- from sensors
- from logic analyzer

Our system compares:

## **Expected vs Observed**

and detects faults.

---

## **WHY THIS IS CALLED AI-BASED**

Because:

- instead of manually writing many if-else conditions,
  - the model learns patterns automatically from data.
- 

## **Decision Tree Simple Idea**

Decision tree works like:

Is Sum wrong?

YES →

Is Sum = 1?

YES → Sum Stuck-at-1

It creates intelligent decision rules automatically.

